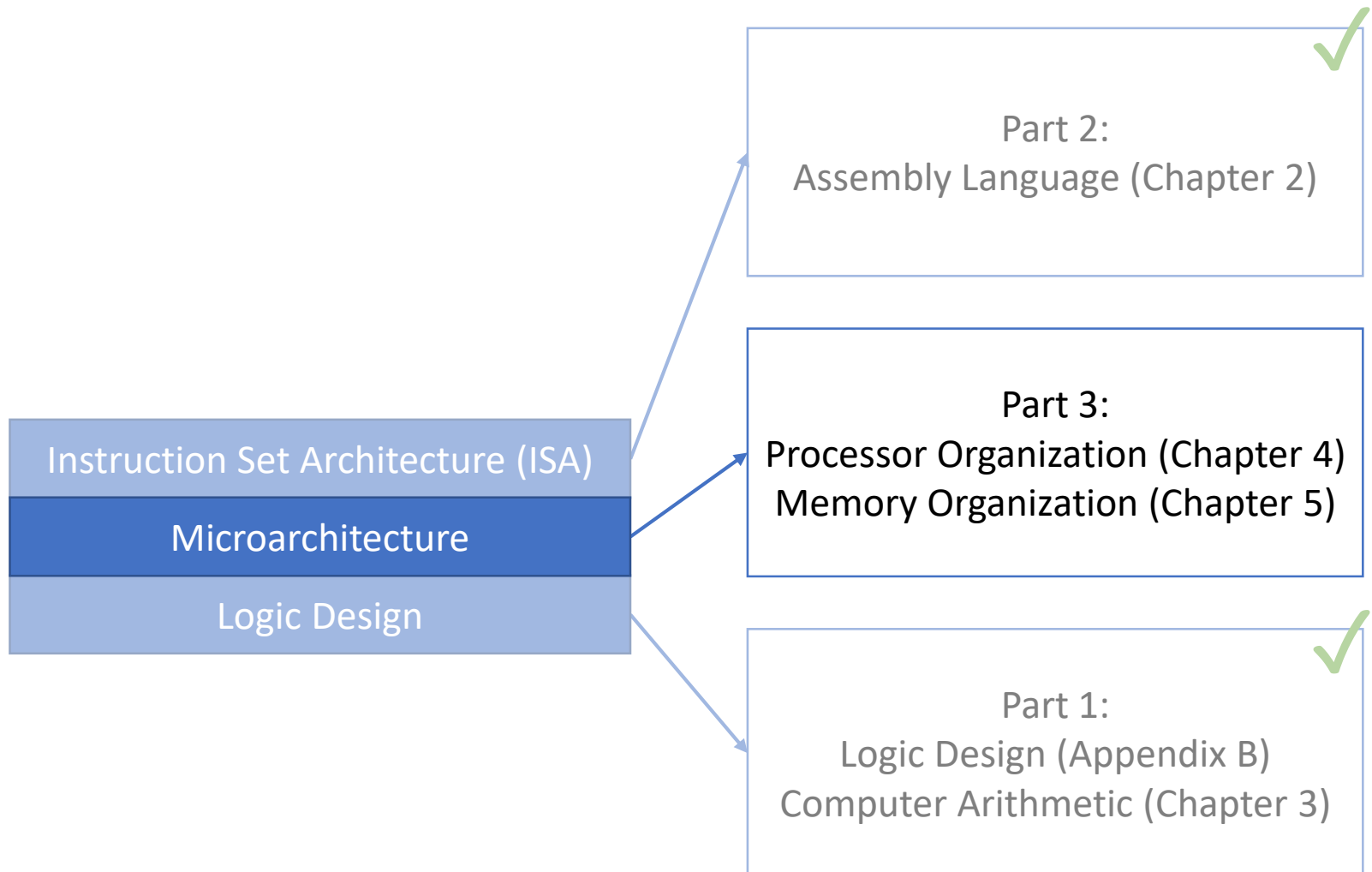


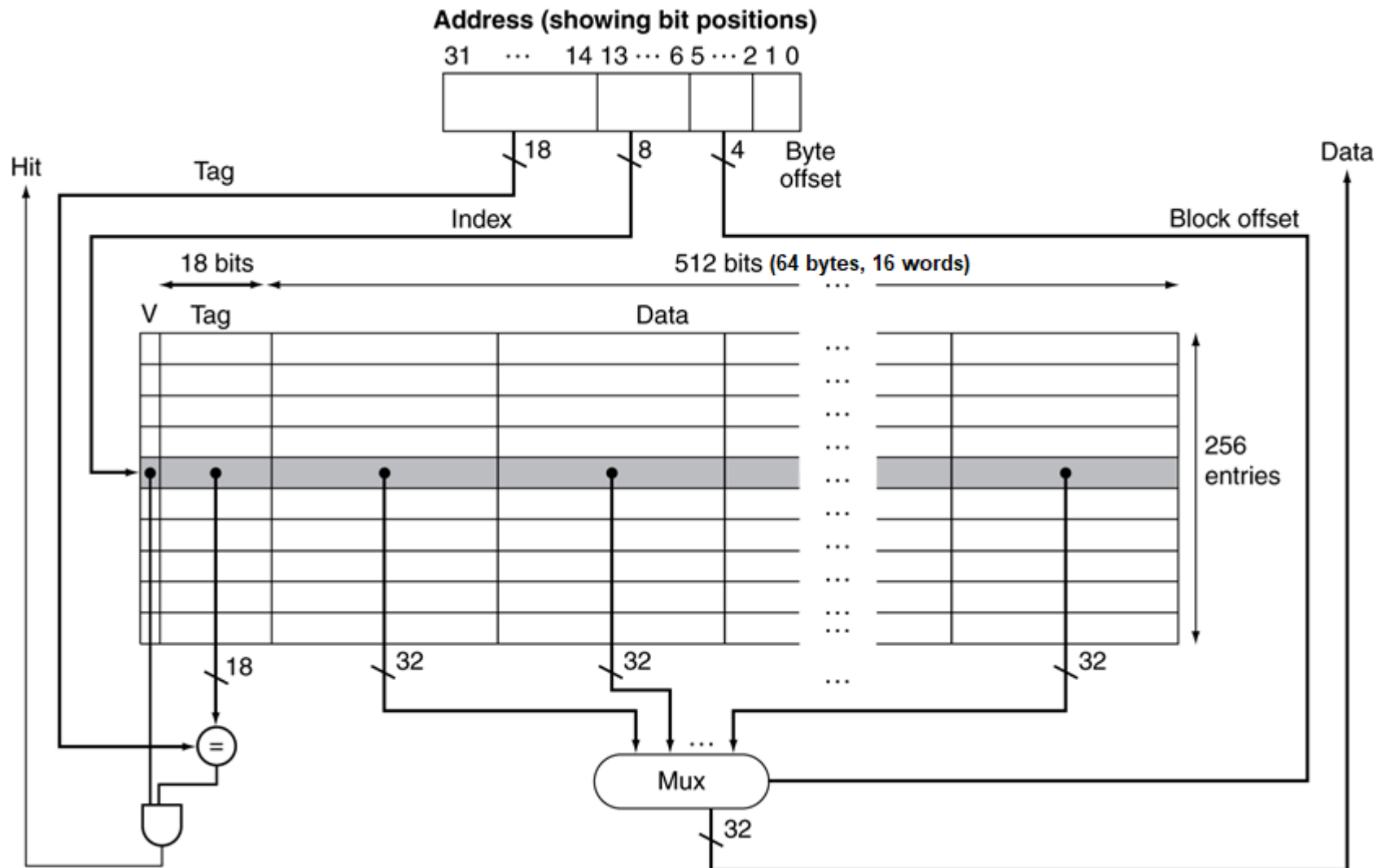
Lecture 30: Associative Caches

CMPS 221 – Computer Organization and Design

Last Time: Memory and Cache



Caches with Large Block Size



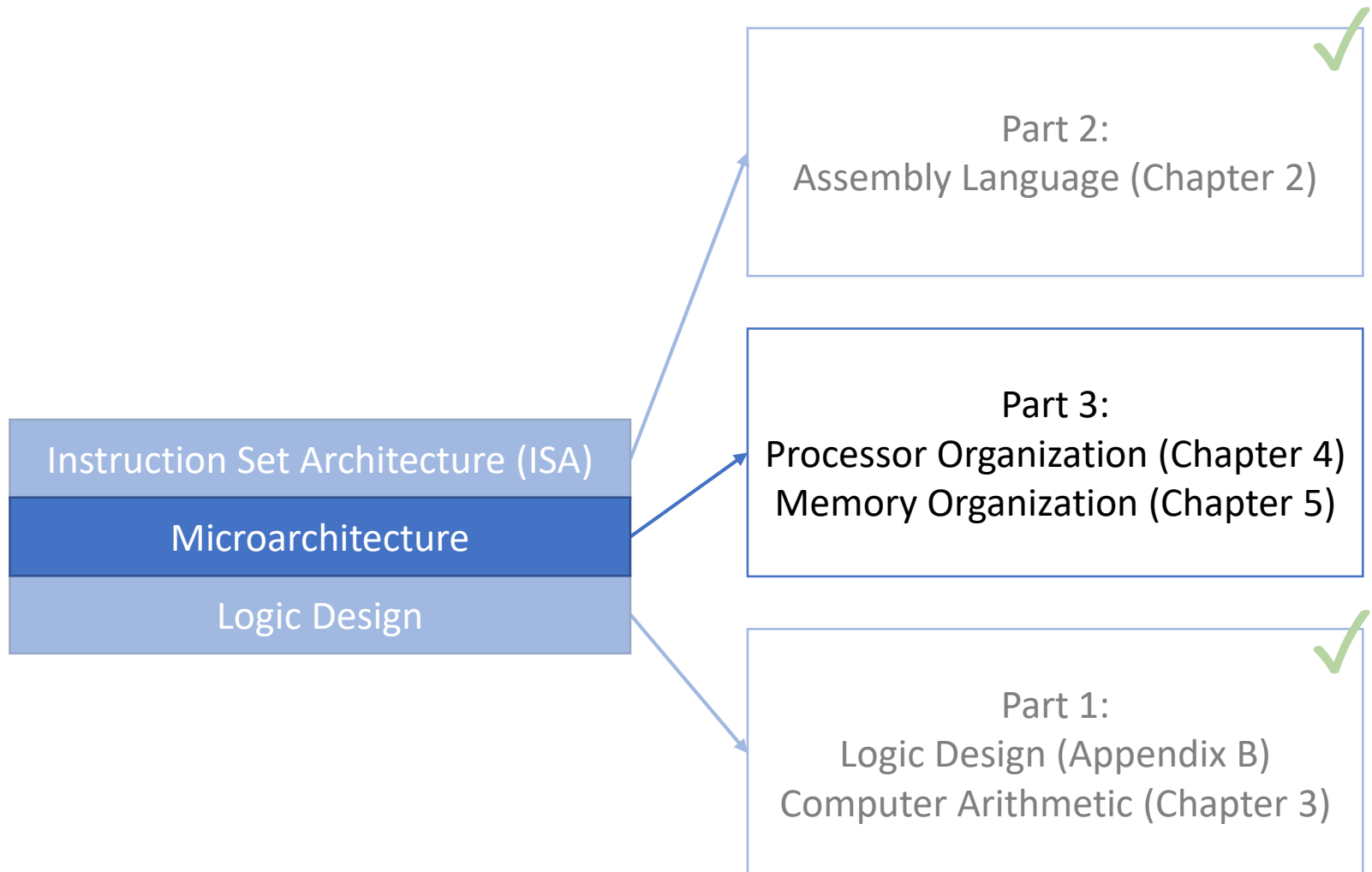
Impact of Block Size

- Advantages of larger blocks:
 - Amortize the overhead of the tag bits
 - Reduces miss rate due to spatial locality
- Disadvantages (assuming fixed-size cache):
 - Fewer blocks (increases miss rate)
 - Underutilizes blocks (pollution) if no spatial locality
 - Larger miss penalty (more bytes to fetch)
 - Can override benefit of reduced miss rate
 - Early restart and critical-word-first can help

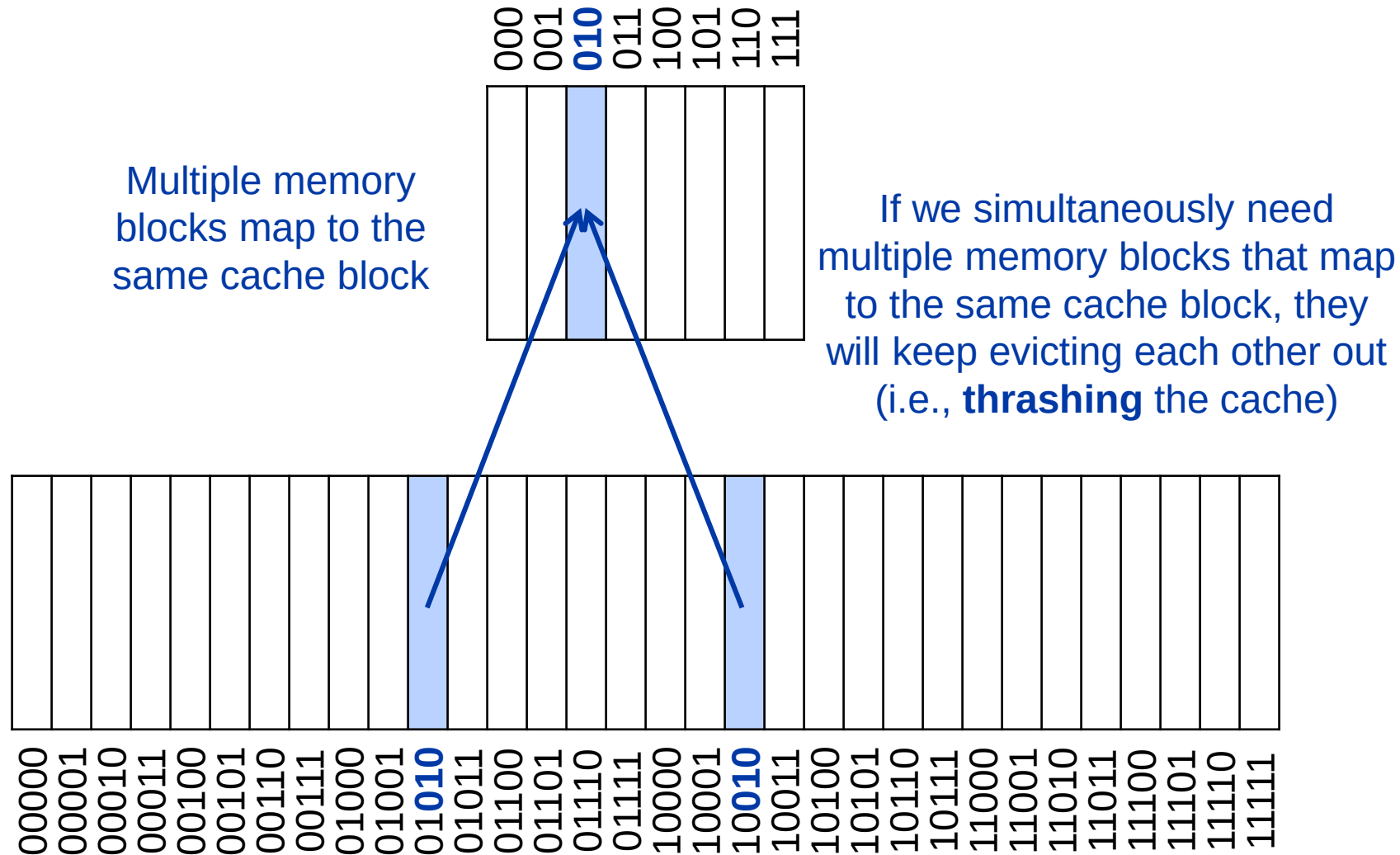
Writing to Caches Summary

- On a **write hit**:
 - A **write-through cache** updates memory
 - A **write-back cache** only updates the block in the cache (and sets a **dirty bit**)
 - Use a **write buffer** to avoid waiting for write-throughs or write-backs to complete
- On a **write miss**:
 - **Write-allocate**: fetch the block
 - **Write-around**: do not fetch the block

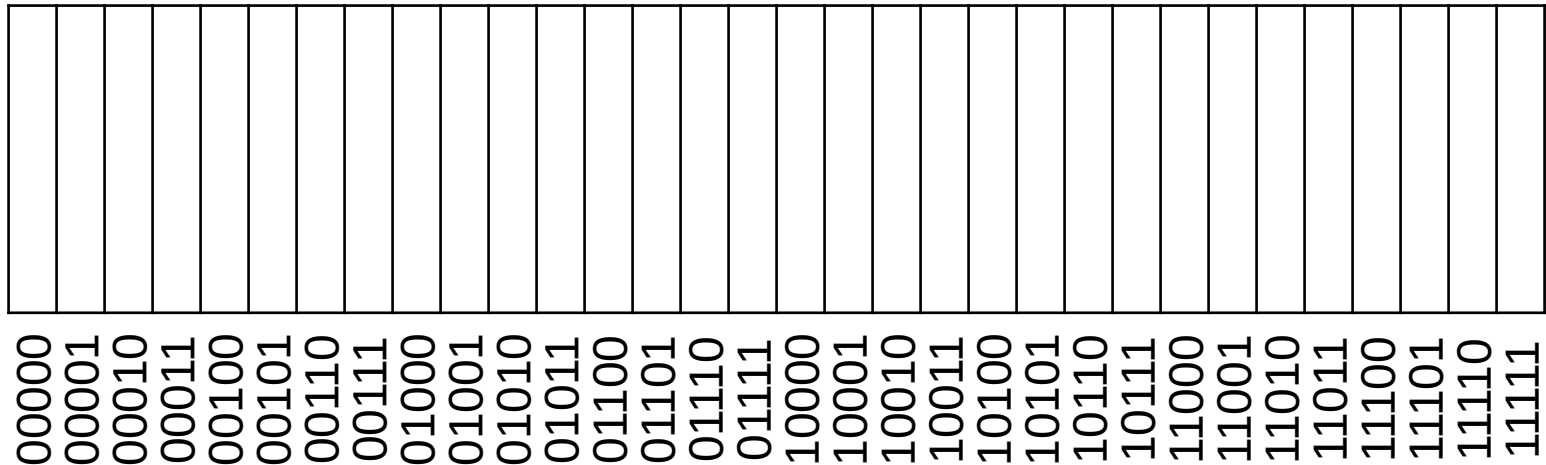
Today: Associative Caches



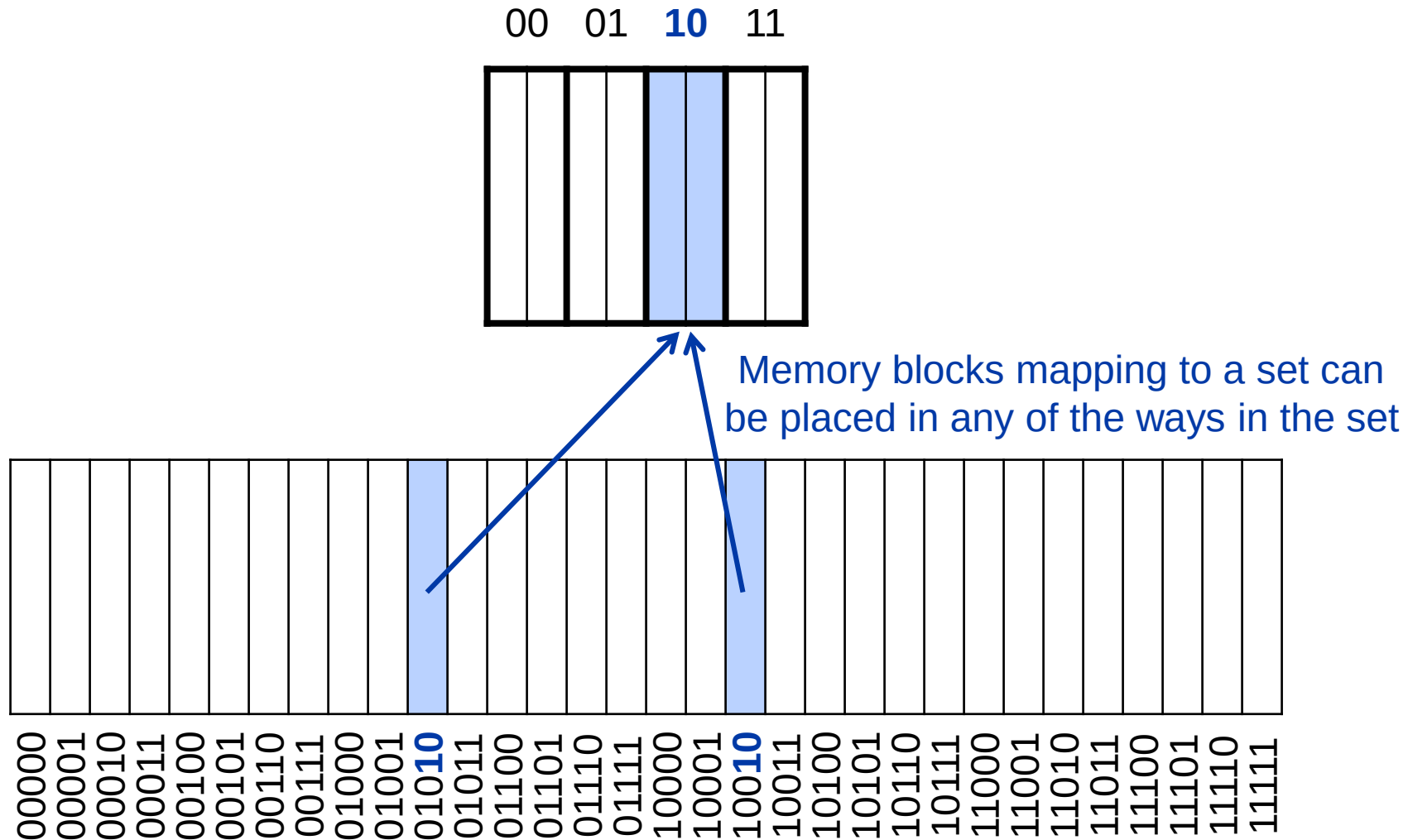
Recall: Direct Mapped Cache



00 01 10 11



Associative Caches



Associative Caches

- **N -way set associative cache**
 - Each set contains N blocks
 - Search all entries in a given set at once
- **Fully associative cache**
 - A block can go in any cache entry
 - The whole cache is one set
 - Search all entries in the cache at once

Spectrum of Associativity

■ Organizations of a cache with 8 blocks

**One-way set associative
(direct mapped)**

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

Address Subdivision

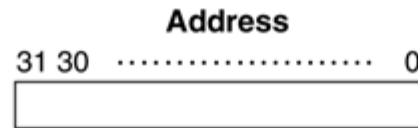
Consider a 4-way set
associative cache
with 2^8 (256) sets

Index	V	Tag	Data	V	Tag	Data	V	Tag	Data	V	Tag	Data
0												
1												
2												
...												
253												
254												
255												

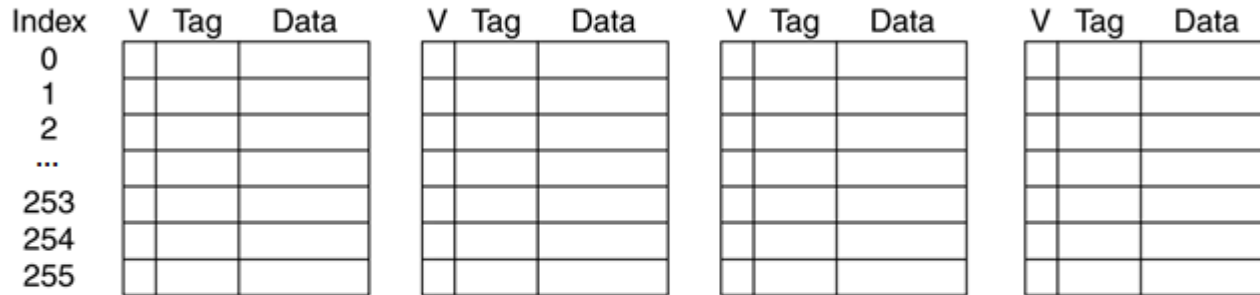
↔
Each block is
1 word (4 bytes)

Address Subdivision

Consider a 4-way set
associative cache
with 2^8 (256) sets

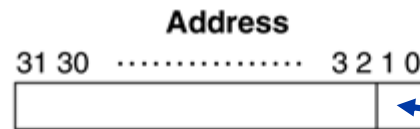


How do we subdivide a
32-bit memory address to
access the cache?



↔
Each block is
1 word (4 bytes)

Address Subdivision

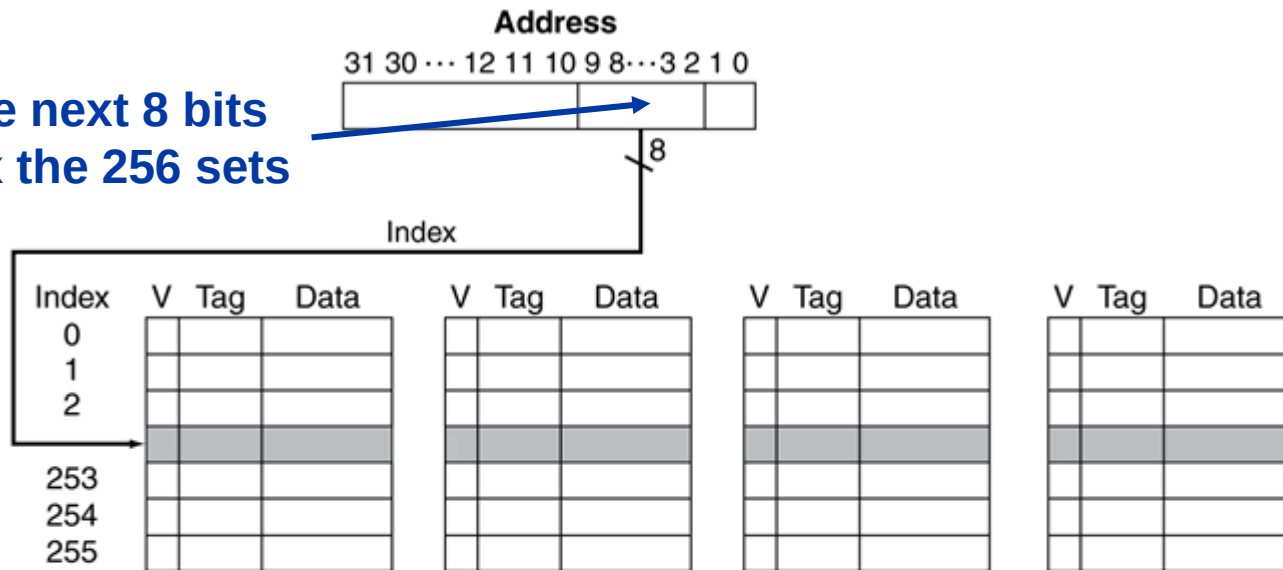


**Lowest 2 address bits
differentiate bytes in the
same word/block**

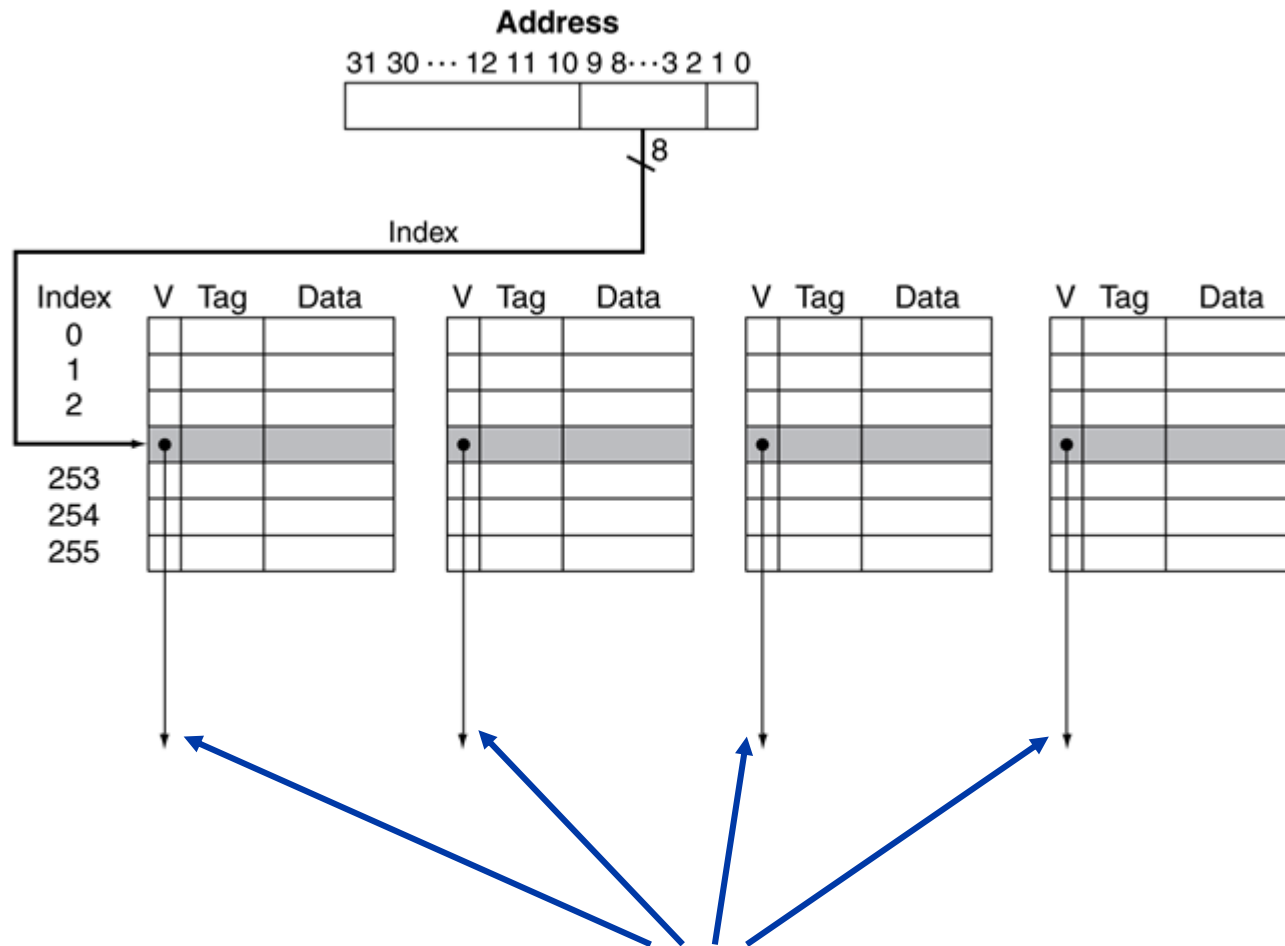
Index	V	Tag	Data	V	Tag	Data	V	Tag	Data	V	Tag	Data
0												
1												
2												
...												
253												
254												
255												

Address Subdivision

Use the next 8 bits
to index the 256 sets



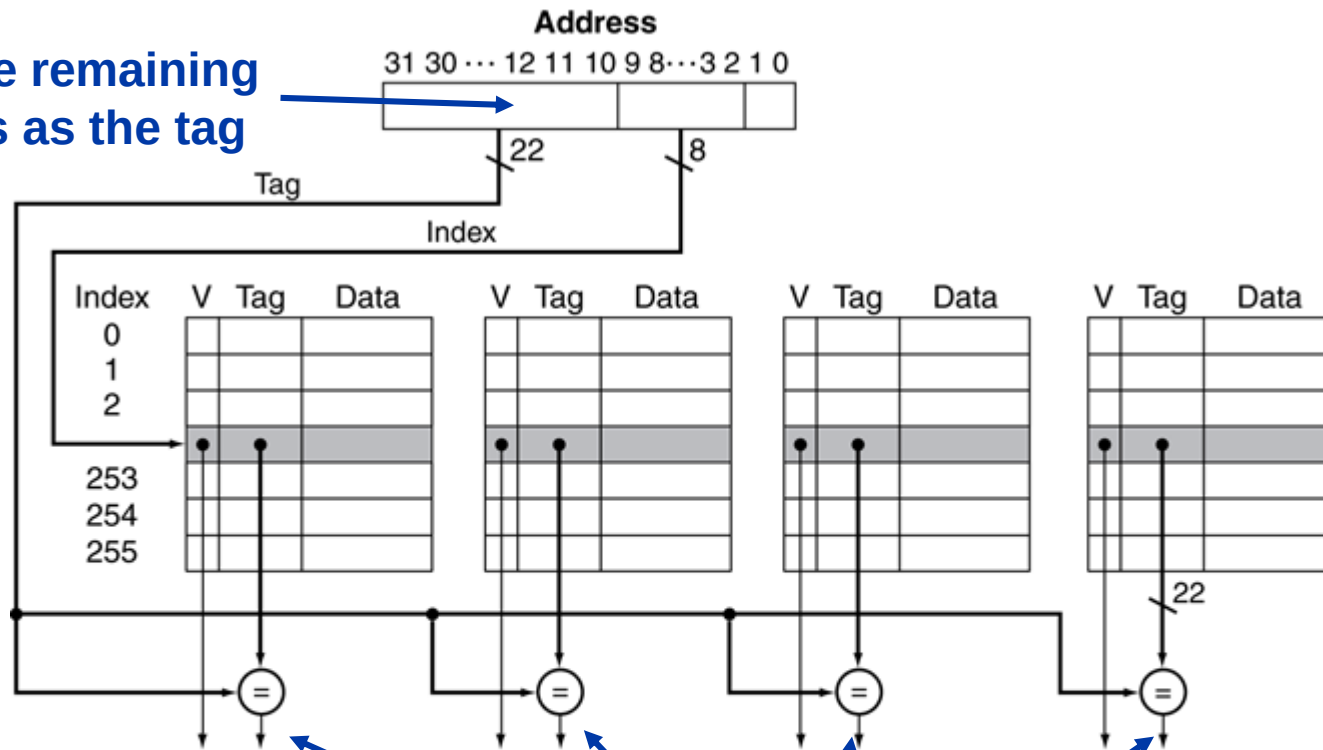
Address Subdivision



Check valid bits to see if any
of the ways are not empty

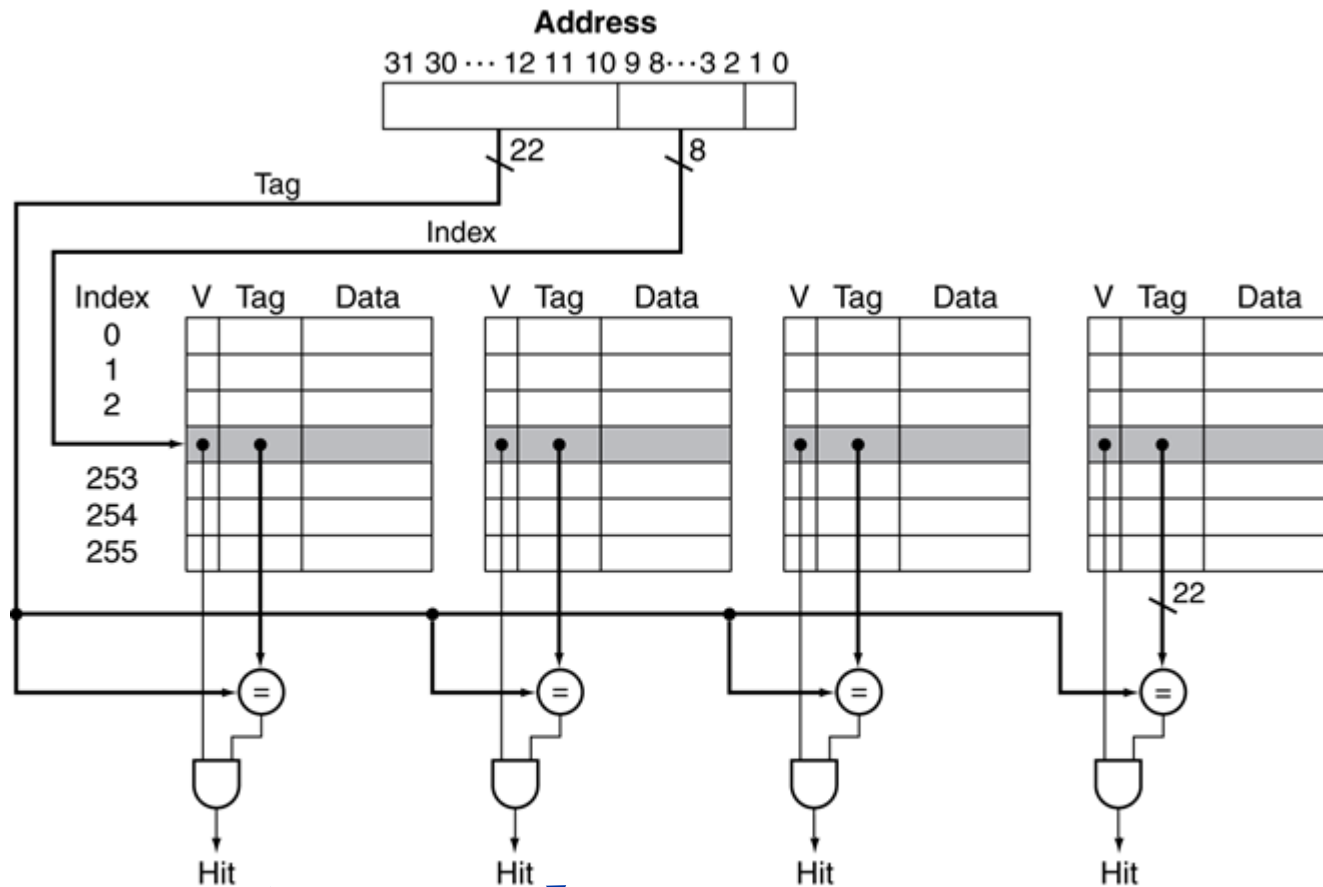
Address Subdivision

Use the remaining
22 bits as the tag



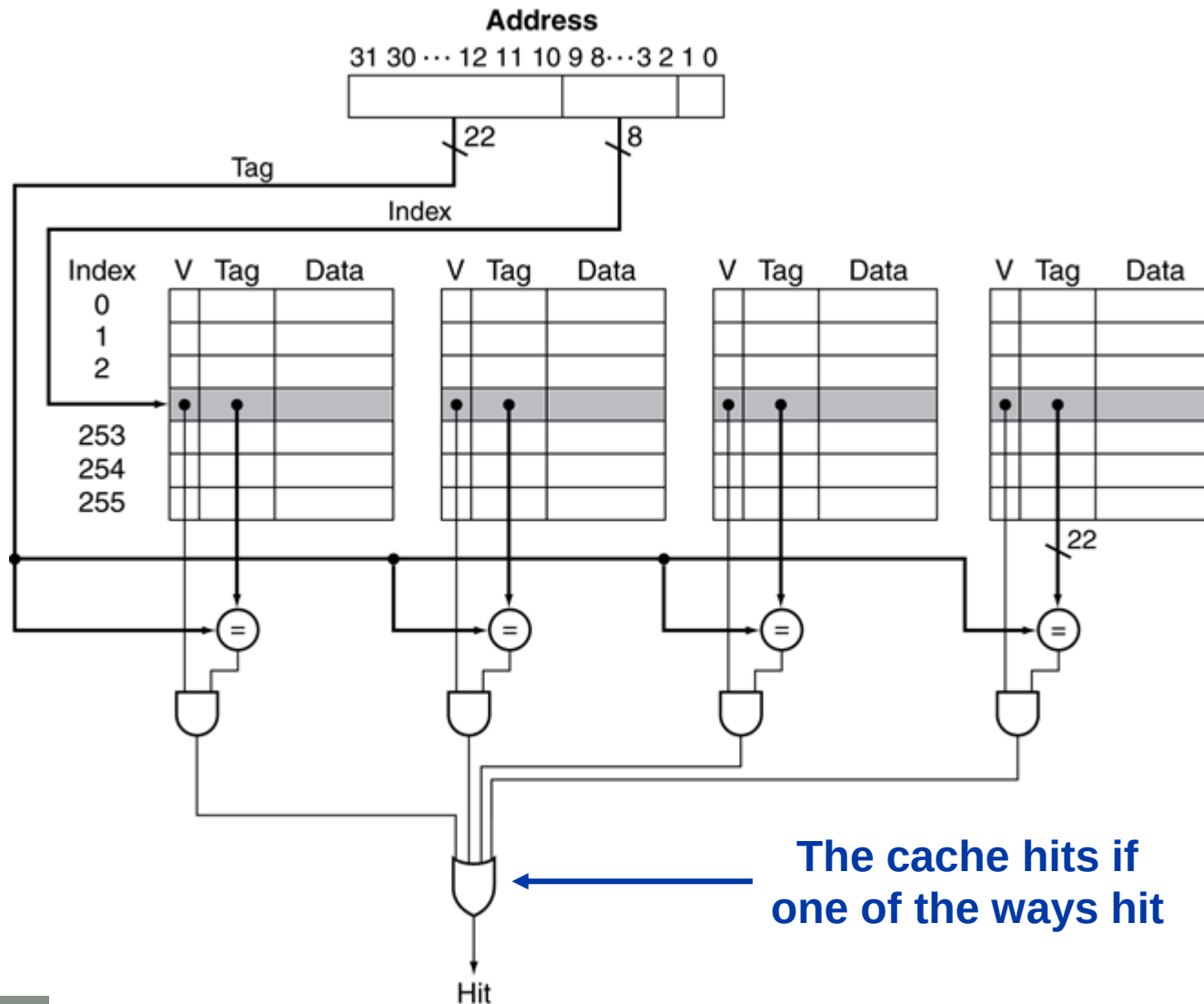
Compare tags to check if
any of the ways match

Address Subdivision

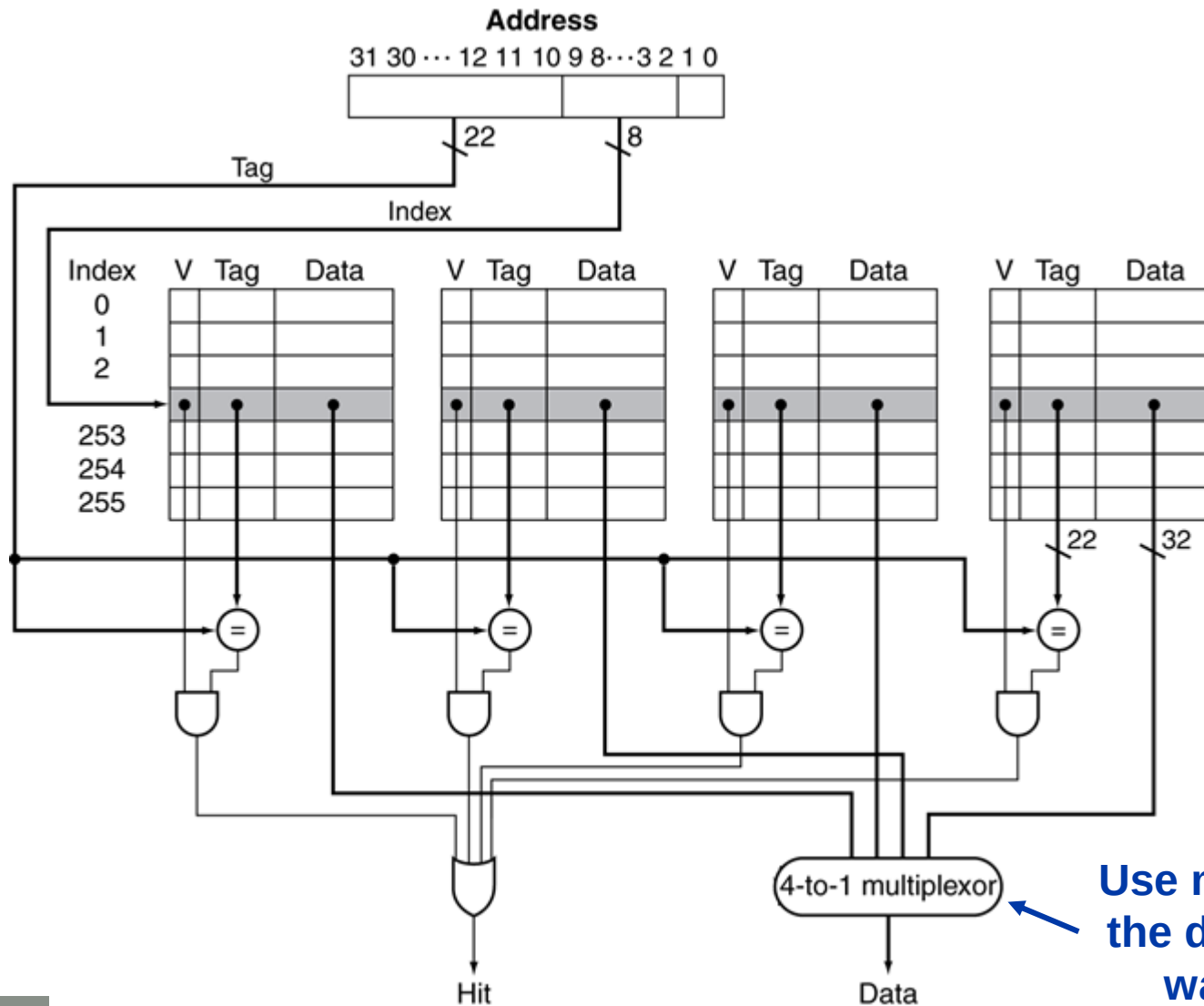


A way hits if its valid bit is set and the tag matches

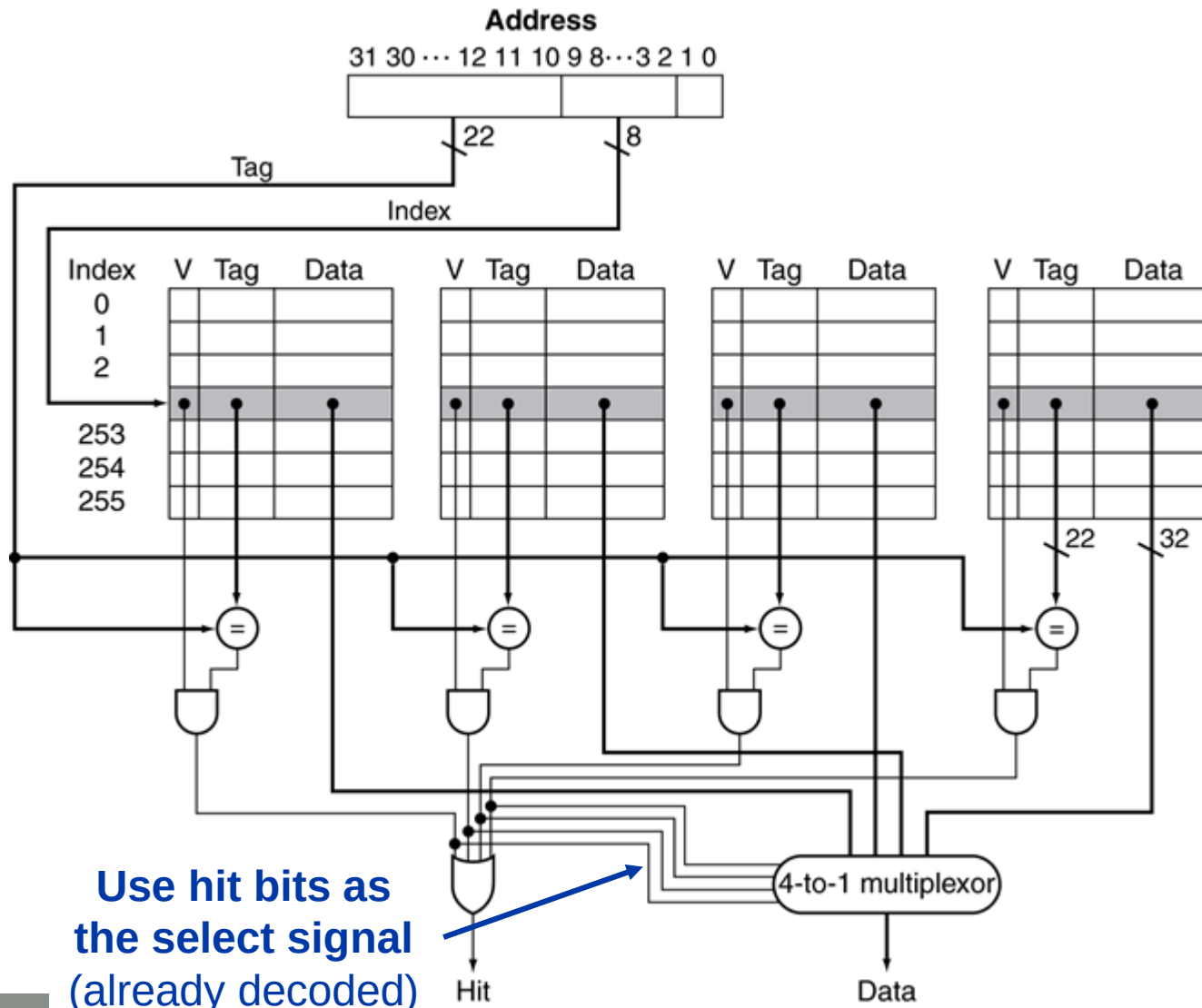
Address Subdivision



Address Subdivision



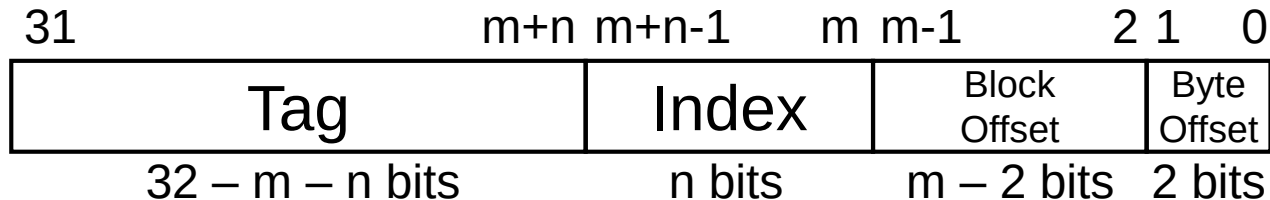
Address Subdivision



Use hit bits as
the select signal
(already decoded)

Bits in an Associative Write-Back Cache

- Cache with 2^n sets, 2^m bytes/block, w ways



- Size of an associative write-back cache
 - Number of blocks = $2^n \cdot w$
 - Valid bits: $2^n \cdot w$
 - Dirty bits: $2^n \cdot w$
 - Tag bits: $2^n \cdot w \cdot (32 - m - n)$
 - Total bits = $2^n \cdot w \cdot (2 + [32 - m - n] + 2^m \cdot 8)$

Associative Cache Example

- Consider a 2-way set associative cache with 4 sets and 8 bytes (2 words) per block for a memory with 8-bit addresses

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00									
01									
10									
11									

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000				

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	0				0				
10	0				0				
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	0				0				
10	0				0				
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	0				0				
10	1	101	x	x	0				
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100				

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	0				0				
10	1	101	x	x	0				
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	0				0				
10	1	101	x	x	0				
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	1	111	x	x	0				
10	1	101	x	x	0				
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss
10010000				

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	1	111	x	x	0				
10	1	101	x	x	0				
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss
10010000	100	10	0	

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	1	111	x	x	0				
10	1	101	x	x	0				
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss
10010000	100	10	0	Miss

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	1	111	x	x	0				
10	1	101	x	x	1	100	x	x	
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss
10010000	100	10	0	Miss
10110100				

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	1	111	x	x	0				
10	1	101	x	x	1	100	x	x	
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss
10010000	100	10	0	Miss
10110100	101	10	1	

Way 0				
Index	V	Tag	Word 0	Word 1
00	0			
01	1	111	x	x
10	1	101	x	x
11	0			

Way 1				
Index	V	Tag	Word 0	Word 1
00	0			
01	0			
10	1	100	x	x
11	0			

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss
10010000	100	10	0	Miss
10110100	101	10	1	Hit

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	1	111	x	x	0				
10	1	101	x	x	1	100	x	x	
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss
10010000	100	10	0	Miss
10110100	101	10	1	Hit
11010000				

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	1	111	x	x	0				
10	1	101	x	x	1	100	x	x	
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss
10010000	100	10	0	Miss
10110100	101	10	1	Hit
11010000	110	10	0	

Way 0

Index	V	Tag	Word 0	Word 1
00	0			
01	1	111	x	x
10	1	101	x	x
11	0			

Way 1

Index	V	Tag	Word 0	Word 1
00	0			
01	0			
10	1	100	x	x
11	0			

Replacement Policy

- A **replacement policy** selects which block to evict if there is no empty block available
- Direct mapped cache: only one choice
- Set associative cache
 - **Least-recently used** (LRU)
 - Choose the one unused for the longest time
 - Simple for 2-way, manageable for 4-way, too hard beyond that
 - **Random**
 - Gives approximately the same performance as LRU for high associativity

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss
10010000	100	10	0	Miss
10110100	101	10	1	Hit
11010000	110	10	0	Miss

Way 0

Index	V	Tag	Word 0	Word 1
00	0			
01	1	111	x	x
10	1	101	x	x
11	0			

Way 1

Index	V	Tag	Word 0	Word 1
00	0			
01	0			
10	1	100	x	x
11	0			

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss
10010000	100	10	0	Miss
10110100	101	10	1	Hit
11010000	110	10	0	Miss

Way 0					Way 1				
Index	V	Tag	Word 0	Word 1	V	Tag	Word 0	Word 1	
00	0				0				
01	1	111	x	x	0				
10	1	101	x	x	1	100	x	x	
11	0				0				

Associative Cache Example

Address	Tag	Index	Block Offset	Hit or Miss
10110000	101	10	0	Miss
11101100	111	01	1	Miss
10010000	100	10	0	Miss
10110100	101	10	1	Hit
11010000	110	10	0	Miss

Way 0

Index	V	Tag	Word 0	Word 1
00	0			
01	1	111	x	x
10	1	101	x	x
11	0			

Way 1

Index	V	Tag	Word 0	Word 1
00	0			
01	0			
10	1	110	x	x
11	0			

How Much Associativity

- Advantage of increasing associativity
 - Decreases miss rate
 - But with diminishing returns
 - Example: Simulation of a system with 64KB D-cache, 16-word blocks, SPEC2000 benchmark suite
 - 1-way: 10.3%
 - 2-way: 8.6%
 - 4-way: 8.3%
 - 8-way: 8.1%
- Disadvantage of increasing associativity
 - Increases hit time
 - More hardware (comparators, mux, etc.)

Textbook Sections

- The content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - 5.4